

Variadic Functions & Command-Line Arguments

Submitted By

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

Contents

This assignment contains two C programs. Each one has the problem statement, the approach used, the complete source code, the sample output and a short explanation.

- **Question 1:** Weighted average using a variadic function (stdarg.h)
- **Question 2:** Statistics of integers given as command-line arguments
- **Conclusion:** Summary of concepts used

Question 1: Weighted Average using a Variadic Function

1.1 Problem Statement

Write a C program containing a variadic function `double weightedAverage(int count, ...)`. The function accepts count followed by count pairs of values (a value and its weight) and returns the weighted average. For example, `weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)` uses the pairs (80.0, 0.3), (75.0, 0.2) and (90.0, 0.5).

The program must validate that all weights are non-negative, handle the case where the total weight is zero, display an appropriate error message for invalid input and use `stdarg.h` correctly.

1.2 Formula

$$\text{Weighted Average} = (v1 \times w1 + v2 \times w2 + \dots + vn \times wn) / (w1 + w2 + \dots + wn)$$

For the sample input: $(80 \times 0.3 + 75 \times 0.2 + 90 \times 0.5) / (0.3 + 0.2 + 0.5) = (24 + 15 + 45) / 1.0 = 84.00$

1.3 Approach / Algorithm

1. Check that count is a positive number; otherwise print an error and stop.
2. Declare a `va_list` variable and initialise it with `va_start(args, count)`.
3. Repeat count times: read the value and the weight using `va_arg(args, double)`.
4. If a weight is negative, print an error message, call `va_end` and return NAN.
5. Otherwise add `value × weight` to `weightedSum` and `weight` to `totalWeight`.
6. After the loop call `va_end`. If `totalWeight` is 0, print an error message and return NAN.
7. Return `weightedSum / totalWeight`.

Note: In a variadic function every floating-point argument is promoted to double, so `va_arg` must always be used with double (never float). Because the function returns a double, the special value NAN (from `math.h`) is used to signal an error, and the caller checks it with `isnan()`.

1.4 Source Code

```
/*
 * Program 1 : Weighted Average using a Variadic Function
 * Name      : Rudra Pratap Singh
 * SAP ID    : 590032811
 * Batch     : 16
 */

#include <stdio.h>
#include <stdarg.h>
#include <math.h>      /* NAN, isnan() */

/*
 * weightedAverage()
 * count : number of (value, weight) pairs that follow
 * ...   : count pairs of doubles -> value1, weight1, value2, weight2, ...
 * Returns the weighted average, or NAN if the input is invalid.
 */
double weightedAverage(int count, ...)
{
    va_list args;
    double value, weight;
    double weightedSum = 0.0;
    double totalWeight = 0.0;
    int i;

    /* Validate the number of pairs */
    if (count <= 0)
    {
        printf("Error: count must be a positive integer (received %d).\n", count);
        return NAN;
    }

    va_start(args, count);      /* start reading after 'count' */

    for (i = 1; i <= count; i++)
    {
        value = va_arg(args, double); /* first of the pair */
        weight = va_arg(args, double); /* second of the pair */

        /* Validate: weight must not be negative */
        if (weight < 0.0)
        {
            printf("Error: weight %.2f of pair %d is negative.\n", weight, i);
            va_end(args);      /* always clean up before returning */
            return NAN;
        }

        weightedSum += value * weight;
        totalWeight += weight;
    }

    va_end(args);      /* end of variable argument processing */

    /* Handle the case where the total weight is zero */
    if (totalWeight == 0.0)
    {
        printf("Error: total weight is zero, weighted average is undefined.\n");
        return NAN;
    }

    return weightedSum / totalWeight;
}

/* Helper to print a result only when the calculation was successful */
void showResult(double result)
```

```

{
    if (!isnan(result))
        printf("Weighted Average = %.2f\n", result);
}

int main(void)
{
    double result;

    /* Test 1 : the sample input from the question */
    printf("Test 1: weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)\n");
    result = weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5);
    showResult(result);

    /* Test 2 : a negative weight (invalid) */
    printf("\nTest 2: weightedAverage(2, 70.0, 0.5, 60.0, -0.5)\n");
    result = weightedAverage(2, 70.0, 0.5, 60.0, -0.5);
    showResult(result);

    /* Test 3 : all weights are zero (total weight zero) */
    printf("\nTest 3: weightedAverage(2, 50.0, 0.0, 90.0, 0.0)\n");
    result = weightedAverage(2, 50.0, 0.0, 90.0, 0.0);
    showResult(result);

    /* Test 4 : invalid count */
    printf("\nTest 4: weightedAverage(0)\n");
    result = weightedAverage(0);
    showResult(result);

    return 0;
}

```

1.5 Output

The program is compiled with `cc q1.c -o q1 -lm` and run as `./q1`. Test 1 is the sample input from the question; Tests 2 to 4 check the error handling.

Terminal output:

```
Test 1: weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)
Weighted Average = 84.00

Test 2: weightedAverage(2, 70.0, 0.5, 60.0, -0.5)
Error: weight -0.50 of pair 2 is negative.

Test 3: weightedAverage(2, 50.0, 0.0, 90.0, 0.0)
Error: total weight is zero, weighted average is undefined.

Test 4: weightedAverage(0)
Error: count must be a positive integer (received 0).
```

1.6 Explanation of the Output

- **Test 1:** Valid input, so the result is $(24 + 15 + 45) / 1.0 = 84.00$, matching the expected answer.
- **Test 2:** The second weight is -0.5 , so validation fails and an error message is displayed.
- **Test 3:** All weights are zero, so the total weight is 0 and division is avoided; an error message is displayed.
- **Test 4:** count is 0, which is invalid, so an error message is displayed.

Question 2: Command-Line Integer Statistics

2.1 Problem Statement

Write a C program that accepts any number of command-line arguments representing integers. The program must validate each argument and ignore non-integer values, display the valid integers in ascending order, and display the minimum, maximum, average, median and second-largest distinct value. User-defined functions must be used for validation, sorting and calculation.

2.2 Approach / Algorithm

1. Read the arguments through `argc` and `argv`. If there are none, print a usage message.
2. Check each argument with `isValidInteger()`. It uses `strtol()` and rejects text such as `abc`, `12abc`, `3.5` and numbers too large for an `int`. Rejected arguments are reported as “Invalid argument ignored”.
3. Store the valid integers in a dynamically allocated array (`malloc`).
4. Sort the array in ascending order with `sortAscending()` (bubble sort) and display it.
5. Minimum is the first element and maximum is the last element of the sorted array.
6. Average = sum of all elements / number of elements.
7. Median is the middle element for an odd count, or the mean of the two middle elements for an even count.
8. Second-largest distinct value: move backwards from the end of the sorted array and pick the first element that differs from the maximum.

2.3 User-Defined Functions

- **`isValidInteger()`** checks whether a string is a valid integer.
- **`sortAscending()`** sorts the array in ascending order.
- **`displayArray()`** prints the array on one line.
- **`calculateAverage()`** returns the mean of the array.
- **`calculateMedian()`** returns the median of the sorted array.
- **`secondLargestDistinct()`** finds the second-largest distinct value, if it exists.

2.4 Source Code

```
/*
 * Program 2 : Command-Line Integer Statistics
 * Name      : Rudra Pratap Singh
 * SAP ID    : 590032811
 * Batch     : 16
 *
 * Compile   : cc program.c -o program
 * Run       : ./program 12 5 abc 8 12 20 3
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <limits.h>

/* Returns 1 if 'str' is a valid integer that fits in an int, else 0.
   If valid, the number is stored in *out. */
int isValidInteger(const char *str, int *out)
{
    char *end;
    long num;

    if (str == NULL || *str == '\\0')
        return 0; /* empty string */

    errno = 0;
    num = strtol(str, &end, 10);

    if (end == str || *end != '\\0')
        return 0; /* no digits or extra characters (e.g. "12abc",
"3.5") */
    if (errno == ERANGE || num > INT_MAX || num < INT_MIN)
        return 0; /* out of int range */

    *out = (int)num;
    return 1;
}

/* Sorts arr[0..n-1] in ascending order (bubble sort) */
void sortAscending(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp      = arr[j];
                arr[j]     = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

/* Prints the array on one line */
void displayArray(const int arr[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d%s", arr[i], (i < n - 1) ? " " : "");
    printf("\\n");
}
```



```

/* Average of the elements */
double calculateAverage(const int arr[], int n)
{
    long long sum = 0;
    int i;
    for (i = 0; i < n; i++)
        sum += arr[i];
    return (double)sum / n;
}

/* Median of an already SORTED array */
double calculateMedian(const int arr[], int n)
{
    if (n % 2 == 1)
        return arr[n / 2];
    return ((double)arr[n / 2 - 1] + arr[n / 2]) / 2.0;
}

/* Second-largest distinct value of a SORTED array.
Returns 1 and stores it in *result if it exists, otherwise 0. */
int secondLargestDistinct(const int arr[], int n, int *result)
{
    int i;
    for (i = n - 2; i >= 0; i--)
    {
        if (arr[i] != arr[n - 1])
        {
            *result = arr[i];
            return 1;
        }
    }
    return 0; /* all values are the same */
}

int main(int argc, char *argv[])
{
    int *numbers;
    int count = 0, i, value, second;

    if (argc < 2)
    {
        printf("Usage: %s <integer1> <integer2> ...\n", argv[0]);
        return 1;
    }

    numbers = (int *)malloc((argc - 1) * sizeof(int));
    if (numbers == NULL)
    {
        printf("Error: memory allocation failed.\n");
        return 1;
    }

    /* Validate every argument; ignore the non-integers */
    for (i = 1; i < argc; i++)
    {
        if (isValidInteger(argv[i], &value))
            numbers[count++] = value;
        else
            printf("Invalid argument ignored: %s\n", argv[i]);
    }

    if (count == 0)
    {
        printf("No valid integers were entered.\n");
        free(numbers);
        return 1;
    }

    sortAscending(numbers, count);

```

```

printf("Valid integers in ascending order: ");
displayArray(numbers, count);

printf("Minimum: %d\n", numbers[0]);
printf("Maximum: %d\n", numbers[count - 1]);
printf("Average: %.2f\n", calculateAverage(numbers, count));
printf("Median: %.2f\n", calculateMedian(numbers, count));

if (secondLargestDistinct(numbers, count, &second))
    printf("Second-largest distinct value: %d\n", second);
else
    printf("Second-largest distinct value: does not exist (all values are
equal)\n");

free(numbers);
return 0;
}

```

2.5 Output

The source file is assumed to be saved as program.c and compiled with `cc program.c -o program`.

Run 1 (sample input): `./program 12 5 abc 8 12 20 3`

```
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
```

Run 2 (extra test, repeated values, decimals, overflow, signs): `./program 7 7 7 x 3.5 9999999999 -4 +9`

```
Invalid argument ignored: x
Invalid argument ignored: 3.5
Invalid argument ignored: 9999999999
Valid integers in ascending order: -4 7 7 7 9
Minimum: -4
Maximum: 9
Average: 5.20
Median: 7.00
Second-largest distinct value: 7
```

Run 3 (no valid integer): `./program hello`

```
Invalid argument ignored: hello
No valid integers were entered.
```

2.6 Explanation of the Output

In Run 1, `abc` is not an integer and is ignored. The valid numbers 12, 5, 8, 12, 20 and 3 sorted give 3 5 8 12 12 20. The minimum is 3 and the maximum is 20. The average is $60 / 6 = 10.00$. The count is even, so the median is $(8 + 12) / 2 = 10.00$. After removing the repeated maximum, the second-largest distinct value is 12.

Run 2 shows that decimals (3.5), plain text (x) and values outside the int range (9999999999) are rejected, while negative (-4) and signed (+9) integers are accepted. When all valid values are equal to the maximum, a second-largest distinct value does not exist, and the program handles that case with a message. Run 3 shows the program does not crash when no valid integer is supplied.

Conclusion

Both programs were compiled with the -Wall -Wextra options without warnings and tested with the sample inputs and extra edge cases. The concepts used are listed below.

- **Question 1:** va_list, va_start, va_arg and va_end from stdarg.h to handle a variable number of arguments.
- **Question 1:** Input validation, error messages and avoiding division by zero.
- **Question 2:** argc and argv, strtol() with error checking, dynamic memory (malloc and free), sorting, and user-defined functions.
- **Question 2:** Calculation of minimum, maximum, average, median and second-largest distinct value.